IV. LESSON PROPER

INFORMATION LEAKS

Information leakage happens whenever a system that is designed to be closed to an eavesdropper reveals some information to unauthorized parties nonetheless.

Mitigation 1: Disable Telltale Server Headers

Make sure to disable any HTTP response headers in your web server configuration that reveal the server technology, language, and version you're running. By default, web servers usually send a Server header back with each response, describing which software is running on the server side. This is great advertising for the web server vendor, but the browser doesn't use it. It simply tells an attacker which vulnerabilities they can probe for. Make sure your web server configuration disables this Server header. (Or if you're feeling mischievous, have it report the wrong web server technology!)

Mitigation 2: Use Clean URLs

When you design your website, avoid telltale file suffixes in URLs, such as *.php*, *.asp*, and *.jsp*. Implement *clean URLs* instead—URLs that do not give away implementation details. URLs with file extensions are common in older web servers, which explicitly reference template filenames. Make sure to avoid such extensions.

Mitigation 3: Use Generic Cookie Parameters

The name of the cookie your web server uses to store session state frequently reveals your server-side technology. For instance, Java web servers usually store the session ID under a cookie named JSESSIONID. Attackers can check these kinds of session cookie names to identify servers, as shown in
Listing 12-1.

```
1 if response.get_cookies.match(/JSESSIONID=(.*);(.*)/i)
jsessionid = $1
post_data = "j_username=#{username}&j_password=#{password}"
response = send_request_cgi({
'uri' => '/admin/j_security_check',
'method' => 'POST',
'content-type' => 'application/x-www-form-urlencoded',
'cookie' => "JSESSIONID=#{jsessionid}",
'data' => post_data,
})
```

Listing 12-1: The hacking tool Metasploit attempting to detect and compromise an Apache
Tomcat server

Note that the Metasploit code checks the name of the session cookie 1. Make sure that your web server sends nothing back in cookies that give clues about your technology stack. Change your configuration to use
generic names for the session cookie (for example, session).

Mitigation 4: Disable Client-Side Error Reporting

Most web servers support *client-side error reporting*, which allows the server to print stack traces and routing information in the HTML of the error page. Client-side error reporting is really useful when debugging errors in test environments. However, stack traces and error logs also tell an attacker which modules or libraries you're using, helping them pick out security vulnerabilities to target. Errors occurring in your data access layer can even reveal details about the structure of your database, which is a major security hazard!

You *must* disable error reporting on the client side in your production environment. You should keep the error page your users see completely generic. At most, users should know that an unexpected error occurred and that someone is looking into the problem. Detailed error reports should be kept in production logs and error reporting tools, which only administrators

can access. Consult your web server's documentation on how to disable client-side error reporting. Listing 12-2 illustrates how you would disable this functionality in a Rails config file.

```
# Full error reports are disabled.
config.consider_all_requests_local = false
```

Listing 12-2: Make sure your production configuration file (typically stored at config /environments/production.rb *in Ruby on Rails) disables client-side error reporting.*

Mitigation 5: Minify or Obfuscate Your JavaScript Files

Many web developers preprocess their JavaScript code before deploying it by using a *minifier*, which takes JavaScript code and outputs a functionally equivalent but highly compressed JavaScript file. Minifiers remove all extraneous characters (such as whitespace) and replace some code statements with shorter, semantically identical statements. A related tool is an *obfuscator*, which replaces method and function names with short, meaningless tokens without changing any behavior in the code, deliberately making the code less readable. The popular UglifyJS utility has both capabilities, and can be invoked directly from the command line with the syntax uglifyjs [*input files*], which makes it straightforward to plug into your build process.

Developers usually minify or obfuscate JavaScript code for performance, because smaller JavaScript files load faster in the browser. This preprocessing also has the positive side effect of making it harder for an attacker to detect which JavaScript libraries you're using. Researchers or attackers periodically discover security vulnerabilities in popular JavaScript libraries

that permit cross-site scripting attacks. Making it harder to detect the libraries you're using will give you more breathing room when exploits are discovered.

Mitigation 6: Sanitize Your Client-Side Files

It's important that you conduct code reviews and use static analysis tools to make sure sensitive data doesn't end up in comments or that dead code doesn't get passed to the client. It's easy for developers to leave comments in HTML files, template files, or JavaScript files that share a little too much information, since we forget that these files get shipped to the browser.

Minifying JavaScript might strip comments, but you need to spot sensitive comments in template files and hand-coded HTML files during code reviews and remove them.

Hacking tools make it easy for an attacker to crawl your site and extract any comments that you've accidentally left behind—hackers often use this technique to scan for private IP addresses accidentally left in comments.
This is often a first port of call when a hacker is attempting to compromise your website.

Stay on Top of Security Advisories

Even with all your security settings locked down, a sophisticated hacker can still make a good guess about the technology you're running. Web servers have telltale behaviors in the way they respond to specific edge cases: deliberately corrupted HTTP requests or requests with unusual HTTP verbs, for example. Hackers can use these unique server-technology fingerprints to identify the server-side technology stack. Even when you follow best practices regarding information leakage, it's important to stay on top of security advisories for the technology you use and deploy patches in a prompt manner.

ENCRYPTION

Encryption in the Internet Protocol

Recall that messages sent over the internet are split into data packets and directed toward their eventual destination via the *Transmission Control Protocol (TCP)*. The recipient computer assembles these TCP packets back into the original message. TCP doesn't dictate *how* the data being sent is meant to be interpreted. For that to happen, both computers need to agree
on how to interpret the data being sent, using a higher-level protocol such as HTTP. TCP also does nothing to disguise the content of the packets being sent. Unsecured TCP conversations are vulnerable to *man-in-themiddle attacks*, whereby malicious third parties intercept and read the packets as they are transmitted.
To avoid this, HTTP conversations between a browser and a web server are secured by *Transport Layer Security (TLS)*, a method of encryption that provides both privacy (by ensuring data packets can't be deciphered by a third party) and *data integrity* (by ensuring that any attempt to tamper with the packets in transit will be detectable). HTTP conversations conducted using TLS are called *HTTP Secure (HTTPS)* conversations. When your web browser connects to an HTTPS website, the browser and web server negotiate which encryption algorithms to use as part of the *TLS handshake*—the exchange of data packets that occurs when a TLS conversation is initiated. To make sense of what happens during the TLS handshake, we need to take a brief detour into the various types of encryption
algorithms. Time for some light mathematics!

Encryption Algorithms, Hashing, and Message Authentication Codes

An *encryption algorithm* takes input data and scrambles it by using an *encryption key*—a secret shared between two parties wishing to initiate secure communication. The scrambled output is indecipherable to anyone without a *decryption key*—the corresponding key required to unscramble the data. The input data and keys are typically encoded as binary data, though the keys may be expressed as strings of text for readability.
Many encryption algorithms exist, and more continue to be invented by mathematicians and security researchers. They can be classified into a few categories: symmetric and

asymmetric encryption algorithms (for ciphering data), hash functions (for fingerprinting data and building other cryptographic algorithms), and message authentication codes (for ensuring
data integrity).

Symmetric Encryption Algorithms

A *symmetric encryption algorithm* uses the same key to encrypt and decrypt data. Symmetric encryption algorithms usually operate as *block ciphers*: they break the input data into fixed-size blocks that can be individually encrypted. (If the last block of input data is undersized, it will be *padded* to fill out the block size.) This makes them suitable for processing streams of
data, including TCP data packets. Symmetric algorithms are designed for speed but have one major security flaw: the decryption key must be given to the receiving party before they decrypt the data stream. If the decryption key is shared over the internet, potential attackers will have an opportunity to steal the key, which allows them to decrypt any further messages. Not good.

Asymmetric Encryption Algorithms

In response to the threat of decryption keys being stolen, *asymmetric encryption algorithms* were developed. Asymmetric algorithms use different keys to encrypt and decrypt data. An asymmetric algorithm allows a piece of software such as a web server to publish its encryption key freely, while keeping its decryption key a secret. Any user agent looking to send secure messages to the server can encrypt those messages by using the server's encryption key, secure in the knowledge that nobody (not even themselves!) will be able to decipher the data being sent, because the decryption key is kept secret. This is sometimes described as *public-key cryptography*: the encryption key (*the public key*) can be published; only the decryption key (the *private key*) needs to be kept secret.

Asymmetric algorithms are significantly more complex and hence slower than symmetric algorithms. Encryption in the Internet Protocol uses a combination of both types, as you will see later in the chapter.

Hash Functions

Related to encryption algorithms are *cryptographic hash functions*, which can be thought of as encryption algorithms whose output *cannot* be decrypted. Hash functions also have a couple of other interesting properties: the output of the algorithm (the *hashed value*) is always a fixed size, regardless of the size of input data; and the chances of getting the same output value, given different input values, is astronomically small. Why on earth would you want to encrypt data you couldn't subsequently decrypt? Well, it's a neat way to generate a "fingerprint" for input data. If you need to check that two separate inputs are the same but don't want to store the raw input values for security reasons, you can verify that both inputs produce the same hashed value. This is how website passwords are typically stored, as we saw in Chapter 8.

When a password is first set by a user, the web server will store the hashed value of the password in the database but will deliberately forget the actual password value. When the user later reenters their password on the site, the server will recalculate the hashed value and compare it with the stored hashed value. If the two hashed values differ, it

indicates the user entered a different password, which means the credentials should be rejected. In this way, a site can check the correctness of passwords without explicitly knowing each user's password. (Storing passwords in plaintext form is a security hazard: if an attacker compromises the database, they get every user's password.)

Message Authentication Codes

Message authentication code (MAC) algorithms are similar to (and generally built on top of) cryptographic hash functions, in that they map input data of an arbitrary length to a unique, fixed-sized output. This output is itself called a *message authentication code*. MAC algorithms are more specialized than hash functions, however, because recalculating a MAC requires a secret key. This means that only the parties in possession of the secret key can generate or check the validity of message authentication codes. MAC algorithms are used to ensure that the data packets transmitted on the internet cannot be forged or tampered with by an attacker. To use a MAC algorithm, the sending and receiving computers exchange a shared, secret key—usually as part of the TLS handshake. (The secret key will itself be encrypted before it is sent, to avoid the risk of it being stolen.) From that point onward, the sender will generate a MAC for each data packet being sent and attach the MAC to the packet. Because the recipient computer has the same key, it can recalculate the MAC from the message. If the calculated MAC differs from the value attached to the packet, this is evidence that the packet has been tampered with or corrupted in some form, or it was not sent by the original computer. Hence, the recipient rejects the data packet.

If you've gotten to this point and are still paying attention, congratulations! Cryptography is a large, complex subject that has its own particular jargon. Understanding how it fits into the Internet Protocol requires balancing multiple concepts in your head at once, so thank you for your patience.

Let's see how the various types of cryptographic algorithms we have discussed are used by TLS.

The TLS Handshake

TLS uses a combination of cryptographic algorithms to efficiently and safely pass information. For speed, most data packets passed over TLS will be encrypted using a symmetric encryption algorithm commonly referred to as the *block cipher*, since it encrypts "blocks" of streaming information. Recall that symmetric encryption algorithms are vulnerable to having their encryption keys stolen by malicious users eavesdropping on the conversation. To safely pass the encryption/decryption key for the block cipher, TLS will encrypt the key by using an *asymmetric* algorithm before passing it to the recipient. Finally, data packets passed using TLS will be tagged using a message authentication code, to detect if any data has been tampered with. At the start of a TLS conversation, the browser and website perform a *TLS handshake* to determine how they should communicate. In the first stage of the handshake, the browser will list multiple cipher suites that it supports. Let's drill down on what this means

Cipher Suites

A *cipher suite* is a set of algorithms used to secure communication. Under the TLS standard, a cipher suite consists of *three* separate algorithms. The first algorithm, the *key-exchange algorithm*, is an asymmetric encryption algorithm. This is used by communicating computers to exchange secret keys for the second encryption algorithm:

the symmetric block cipher designed for encrypting the content of TCP packets. Finally, the cipher suite specifies a MAC algorithm for authenticating the encrypted messages.
Let's make this more concrete. A modern web browser such as Google Chrome that supports TLS 1.3 offers numerous cipher suites. At the time of writing, one of these suites goes by the catchy name of ECDHE-ECDSAAES128- GCM-SHA256. This particular cipher suite includes ECDHE-RSA as the key-exchange algorithm, AES-128-GCM as the block cipher, and
SHA-256 as the message authentication algorithm. Want some more, entirely unnecessary, detail? Well, *ECDHE* stands for
Elliptic Curve Diffie–Hellman Exchange (a modern method of establishing a shared secret over an insecure channel). *RSA* stands for the *Rivest–Shamir– Adleman* algorithm (the first practical asymmetric encryption algorithm, invented by three mathematicians in the 1970s after drinking a lot of Passover wine). *AES* stands for the *Advanced Encryption Standard* (an algorithm invented by two Belgian cryptographers and selected by the National Institute of Standards and Technology through a three-year review process).
This particular variant uses a 128-bit key in Galois/Counter Mode, which is specified by *GCM* in the name. Finally, SHA-256 stands for the *Secure Hash Algorithm* (a hash function with a 256-bit word size). See what I mean about the complexity of modern encryption standards? Modern browsers and web servers support a fair number of cipher suites, and more get added to the TLS standard all the time. As weaknesses are discovered in existing algorithms, and computing power gets cheaper, security researchers update the TLS standard to keep the internet secure.
As a web developer, it's not particularly important to understand how these algorithms work, but it *is* important to keep your web server software up-todate so you can support the most modern, secure algorithms.

Session Initiation

Let's continue where we left off. In the second stage of the TLS handshake, the web server selects the most secure cipher suite it can support and then instructs the browser to use those algorithms for communication. At the same time, the server passes back a *digital certificate*, containing the server name, the trusted certificate authority that will vouch for the authenticity of the certificate, and the web server's encryption key to be used in the keyexchange algorithm. (We will discuss what certificates are and why they are necessary for secure communication in the next section.)
Once the browser verifies the authenticity of the certificate, the two computers generate a *session key* that will be used to encrypt the TLS conversation with the chosen block cipher. (Note that this session key is different from the HTTP *session identifier* discussed in previous chapters. TLS handshakes occur at a lower level of the Internet Protocol than the HTTP conversation, which has not begun yet.) The session key is a large random number generated by the browser, encrypted with the (public) encryption key attached to the digital certificate using the key-exchange algorithm, and transmitted to the server. Now, finally, the TLS conversation can begin. Everything past this point will be securely encrypted using the block cipher and the shared session identifier, so the data packets will be indecipherable to anyone snooping on the conversation. The browser and server use the agreed-upon encryption algorithm and session key to encrypt packets in both directions. Data packets are also authenticated and tamper-

proof, using message authentication codes.
As you can see, a lot of complex mathematics underpin secure communication on the internet. Thankfully, the steps involved for enabling HTTPS as a web developer are much simpler. Now we have the theory out of the way, let's take a look at the practical steps needed to secure your users.

Enabling HTTPS

Securing traffic for your website is a lot easier than understanding the underlying encryption algorithms. Most modern web browsers are selfupdating; the development teams for each major browser will be on the cutting edge of supporting modern TLS standards. The latest version of your web server software will support similarly modern TLS algorithms.
That means that the only responsibility left to you as a developer is to obtain a digital certificate and install it on your web server. Let's discuss how to do that and illuminate why certificates are necessary.

Digital Certificates

A *digital certificate* (also known as a *public-key certificate*) is an electronic document used to prove ownership of a public encryption key. Digital certificates are used in TLS to associate encryption keys with internet domains (such as *example.com*). They are issued by *certificate authorities*, which act as a trusted third party between a browser and a website, vouching that a given encryption key should be used to encrypt data being sent to the website's domain. Browser software will trust a few hundred certificate authorities— for example, Comodo, DigiCert, and, more recently, the nonprofit Let's Encrypt. When a trusted certificate authority vouches for a key and domain, it assures your browser that it's communicating with the right website using the right encryption key, thereby blocking an attacker from presenting a
malicious website or certificate. You might ask: why is a third party required to exchange encryption keys on the internet? After all, isn't the whole point of asymmetric encryption that the public key can be made available freely by the server itself?
While this statement is true, the actual process of fetching an encryption key on the internet depends on the reliability of the internet's *Domain Name System (DNS)* that maps domain names to IP addresses. Under some circumstances, DNS is vulnerable to *spoofing attacks* that can be used to direct internet traffic away from a legitimate server to an IP address controlled by an attacker. If an attacker can spoof an internet domain, they can issue their own encryption key, and victims would be none the wiser.
Certificate authorities exist to prevent encrypted traffic from being spoofed. Should an attacker find a way to divert traffic from a legitimate (secure) website to a malicious server under their control, that attacker will typically not possess the decryption key corresponding to the website's certificate. This means they will be unable to decrypt intercepted traffic that was encrypted using the encryption key attached to the site's digital certificate.
On the other hand, if the attacker presents an *alternative* digital certificate corresponding to a decryption key that they *do* possess, that certificate will not have been verified by a trusted certificate authority. Any browser visiting the spoofed website will show a security warning to the user, strongly dissuading them from continuing. In this way, certificate authorities allow users to trust the websites they are visiting. You can view the certificate a website is using by clicking the padlock icon in the browser

bar. The information described there won't be particularly interesting, but browsers do a good job of warning you when a certificate is invalid.

Obtaining a Digital Certificate

Obtaining a digital certificate for your website from a certificate authority requires a few steps, by which the authority verifies that you own your domain. The precise way you perform these steps differs depending on which certificate authority you choose. The first step is to generate a *key pair*, a small digital file containing randomly generated public and private encryption keys. Next, you use this key pair to generate a *certificate signing request (CSR)* that contains the public key
and domain name of your website, and upload the request to a certificate authority. Before honoring the signing request and issuing the certificate, the certificate authority will require you to demonstrate to them that you
have control of the internet domain contained in the CSR. Once domain ownership has been verified, you can download the certificate and install it on your web server along with the key pair.

Generating a Key Pair and Certificate Signing Request

The key pair and CSR are typically generated using the command line tool, openssl. CSRs often contain other information about the applicant besides the domain name and public key, such as the organization's legal name and physical location. These get included in the signed certificate, but are not mandatory unless the certificate authority chooses to validate them. During the generation of the signing request, the domain name is often referred to as the *distinguished name (DN)* or the *fully qualified domain name (FQDN)*, for historical reasons. Listing 13-1 shows how to generate a certificate signing
request by using openssl.

```
openssl req -new -key ./private.key -out ./request.csr
```

Listing 13-1: Generating a certificate signing request by using openssl on the command line

The file *private.key* should contain a newly generated private key (which can also be generated with openssl). The tool openssl will ask for details to incorporate into the signing request, including the domain name.

Domain Verification

Domain verification is the process by which a certificate authority verifies that someone applying for a certificate for an internet domain does indeed have control of that domain. When applying for a digital certificate, you are stating that you need to be able to decrypt traffic sent to a particular internet domain. The certificate authority will insist on checking that you own that domain as part of its due diligence.

Domain verification generally requires you to make a temporary edit to the DNS entries for your domain, thus demonstrating that you have edit rights in the DNS. Domain verification is what protects against DNS spoofing attacks: an attacker cannot apply for a certificate unless they also have edit rights.

Extended Validation Certificates

Some certificate authorities issue *extended validation (EV)* certificates. These require the certificate authority to collect and verify information about the legal entity applying

for a certificate. That information will then be included in the digital certificate, and made available in the web browser to users visiting the website. EV certificates are popular with large organizations, because the name of the company is usually displayed alongside the padlock icon in the browser URL bar, encouraging a sense of trust in users.

Expiring and Revoking Certificates

Digital certificates have a finite lifespan (typically in years or months) after which they must be reissued by the certificate authority. Certificate authorities also keep track of certificates that have been voluntarily *revoked* by certificate holders. If the private key corresponding to your digital certificate gets compromised, it's important that you as a site owner apply for a new
certificate and then revoke the prior certificate. Browsers will warn a user when visiting a website with an expired or revoked certificate.

Self-Signed Certificates

For some environments, particularly testing environments, acquiring a certificate from a certificate authority is unnecessary or impractical. Testing environments that are available on only an internal network, for example, can't be verified by a certificate authority. You may still want to support HTTPS on these environments, however, so the solution is to generate your own certificate—a *self-signed certificate*. Command line tools like openssl can easily produce self-signed certificates.

Browsers encountering a site with a self-signed certificate will usually issue a strident security warning to the user (This site's security certificate is not trusted!) but will still allow the user to accept the risks and continue anyway. Just make sure anyone using your test environment is aware of this limitation and knows why the warning occurs.

Should You Pay for Certificates?

Certificate authorities were traditionally commercial entities. Even today, many of them charge a fixed fee for each certificate being issued. Since 2015, the California nonprofit Let's Encrypt has offered free certificates. Let's
Encrypt was founded by (among others) the Mozilla Foundation (which coordinates releases of the Firefox browser) and the Electronic Frontier Foundation (a digital rights nonprofit based in San Francisco). As a result, there is little reason to pay for a certificate, unless you require extended validation capabilities offered by commercial certificate authorities.

Installing a Digital Certificate

Once you have a certificate and a key pair, the next step is to get your web server to switch to using HTTPS and serve the certificate as part of the TLS handshake. This process varies depending on your hosting provider and server technology, though it's normally pretty straightforward and welldocumented.

Let's review a typical deployment process—which will require a short digression.

.

Web Servers vs. Application Servers

Up to this point in the book, I have described web servers as machines for intercepting and answering HTTP requests, and talked about how they either send back static content or execute code in response to each request.

While this is an accurate description, it elides the fact that websites are usually

deployed as a *pair* of running applications.

The first of the applications that runs a typical website is a *web server* that serves static content and performs low-level TCP functions. This will typically be something like Nginx or the Apache HTTP Server. Web servers are written in C and optimized to quickly perform low-level TCP functions. The second application of the pair is an *application server*, which sits downstream from the web server and hosts the code and templates that make up that dynamic content of the site. Many application servers are available for each programming language. A typical application server might be Tomcat or Jetty for websites written in the Java languages; Puma or Unicorn for Ruby on Rails websites; Django, Flask, or Tornado for Python websites; and so on.

Rather confusingly, web developers will often casually refer to the application server they use as "the web server," since that is the environment they spent most of the time writing code for. In actual fact, it's perfectly possible to deploy an application server on its own, because an application server can do everything a web server can, albeit less efficiently. This is a fairly typical setup when a web developer is writing and testing code on their own machine.

Configuring Your Web Server to Use HTTPS

Digital certificates and encryption keys are almost always deployed to web servers, since they are much faster than application servers. Switching over a web server to use HTTPS is a matter of updating the web server's configuration so that it accepts traffic on the standard HTTPS port (443), and telling it the location of the digital certificate and key pair to be used when establishing the TLS session. Listing 13-2 shows how to add the certificate into the configuration file for the Nginx web server.

```
server {
listen 443 ssl;
server_name www.example.com;
ssl_certificate www.example.com.crt;
ssl_certificate_key www.example.com.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;
}
```

*Listing 13-2: Describing the location of the digital certificate (*www.example.com.crt*) and encryption key (*www.example.com.key*) when configuring Nginx*

Web servers that handle TLS functionality in this way will decrypt incoming HTTPS requests, and pass any requests that need to be handled by the application server downstream as unencrypted HTTP requests. This is called *terminating HTTPS* at the web server: traffic between the web and application server is not secure (because the encryption has been stripped), but this isn't usually a security risk because traffic is not leaving the physical machine (or at least, will only be passed over a private network).

What About HTTP?

Configuring your web server to listen for HTTPS requests on port 443 requires a handful of edits to a configuration file. You then need to decide how your web server will treat unencrypted traffic on the standard HTTP port (80). The usual method is to instruct the web server to redirect insecuretraffic to the corresponding secure URL. For instance: if

a user agent
visits *http://www.example.com/page/123*, the web server will respond with an HTTP 301 response, directing the user agent to visit *https://www.example .com/page/123* instead. The browser will understand this as an instruction to send the same request on port 443, after negotiating a TLS handshake.

Listing 13-3 shows an example of how to redirect all traffic on port 80 to
port 443 on the Nginx web server.

```
server {
listen 80 default_server;
server_name _;
return 301 https://$host$request_uri;
}
```
Listing 13-3: Redirecting all HTTP to HTTPS on the Nginx web server

HTTP Strict Transport Security

At this point, your site is set up to securely communicate with the browser, and any browsers using HTTP will get redirected to HTTPS. You have one final loophole to take care of: you need to ensure that sensitive data will not
be sent during any initial connection over HTTP.
When a browser visits a site it has seen previously, the browser sends back any cookies the website previously supplied in the Cookie header of a request. If the initial connection to the website is done over HTTP, that cookie information will be passed insecurely, even if the subsequent requests and responses get upgraded to HTTPS. Your website should instruct browsers to send cookies *only* over an HTTPS connection by implementing an *HTTP Strict Transport Security (HSTS)* policy.
You do this by setting the header Strict-Transport-Security in your responses.
A modern browser encountering this header will remember to connect to your site *only* using HTTPS. Even if the user explicitly types in an HTTP address like *http://www.example.com*, the browser will switch to using HTTPS without being prompted. This protects cookies from being stolen during the initial connection to your site. Listing 13-4 shows how to add a Strict

-Transport-Security header when using Nginx.

```
server {
add_header Strict-Transport-Security "max-age=31536000" always;
}
```
Listing 13-4: Setting up HTTP Strict Transport Security in Nginx

The browser will remember not to send any cookies over HTTP for the number of seconds supplied in max-age, whereupon it will check again if the site has changed its policy.

Attacking HTTP (and HTTPS)

At this point in the chapter, you might well ask: what's the worst that can happen if I choose *not* to use HTTPS? I haven't really described how unencrypted HTTP can be exploited, so let's remedy that. Weakly encrypted or unencrypted communication on the

internet allows an attack to launch a man-in-the-middle attack, whereby they tamper with or snoop on the HTTP conversation. Let's look at some recent examples from hackers, internet service providers, and governments.

Wireless Routers

Wireless routers are a common target for man-in-the-middle attacks. Most routers contain a bare-bones installation of the Linux operating system, which enables them to route traffic to a local *internet service provider (ISP)* and host a simple configuration interface. This is a perfect target for a hacker, because the Linux installation will typically *never* be updated with security patches—and the same operating system version will be installed in many thousands of homes.

In May 2018, Cisco security researchers discovered that over half a million Linksys and Netgear routers had been infected with a piece of malware called *VPNFilter*, which snooped on HTTP traffic passing through the router, stealing website passwords and other sensitive user data on behalf of an unknown attacker thought to be linked to the Russian government.

VPNFilter even attempted to perform *downgrade attacks*, interfering with the initial TLS handshake to popular sites so that the browser opted to use weaker encryption or no encryption at all. Sites using HTTPS would have been immune to this attack, because HTTPS traffic is indecipherable to anyone but the recipient site. Traffic to other websites was likely stolen by hackers and mined for sensitive data.

Wi-Fi Hotspots

A lower-tech way for a hacker to launch a man-in-the-middle attack is to simply set up their own Wi-Fi hotspot in a public place. Few of us pay much attention to the name of the Wi-Fi hotspots our devices use, so it's easy for an attacker to set up a hotspot in a public space like a café or hotel lobby and wait for unwary users to connect to it. Because TCP traffic will flow
through the hacker's device on its way to the ISP, the hacker will be able to record the traffic to disk and comb through it to extract sensitive details like credit card numbers and passwords. The only indication to the victim that anything untoward has happened occurs when the attacker leaves the physical location and shuts down the hotspot, disconnecting their victims from the internet. Encrypting traffic defeats this attack, since the hacker will not be able to read any traffic they captured.

Internet Service Providers

Internet service providers connect individual users and businesses to the internet backbone, which is a position of enormous trust given the potentially sensitive nature of the data being passed. You would think that would deter them from snooping or interfering with HTTP requests, but that isn't the case for companies like Comcast, one of the largest ISPs in the United States, which injected JavaScript advertisements into HTTP traffic flowing through its servers for many years. Comcast claimed to be doing this as a service (many of the advertisements informed the user of how much of the monthly data plan had already been used), but digital rights campaigners saw this approach as analogous to a mail carrier slipping advertising material into sealed letters. Websites that use HTTPS are immune to this type of tampering, because the contents of each request and response are opaque to the ISP.

Government Agencies

Government agencies snooping on your internet traffic might seem like the stuff of conspiracy theories, but plenty of evidence indicates this does indeed happen. The US *National Security Agency (NSA)* has successfully implemented man-in-middle-attacks to conduct surveillance. An internal presentation leaked by former NSA contractor Edward Snowden
described how Brazil's state-run oil producer Petrobras was spied on: the NSA obtained digital certificates for Google websites and then hosted its own look-alike sites that harvested user credentials while proxying traffic to Google. We don't really know how widespread this type of program is, but it's pretty unnerving to think about. (In case anyone from the government is reading this: actually, this type of program is good and keeps us safe, and the author of this book supports it wholeheartedly.)